

Exercicio: Cadastro de Clientes em Arquivo de Registros Variaveis, Ordenado por CPF, com Remocao e Reindexacao

Nivel: dificil | Tema: arquivos binarios em C, registros de tamanho variavel com separadores

Baseado na tecnica de registros com separadores (`cpf|nome|sobrenome|`) mantidos ordenados, implemente um sistema de cadastro de clientes em `clientes.bin` com as seguintes regras.

Estrutura do registro (serializado com delimitador "|")

`cpf|nome|sobrenome|telefone|cidade|`

Requisitos

- 1. Insercao ordenada (ja visto):** Mantenha o arquivo sempre ordenado por CPF, sem duplicatas — reaproveite a logica de merge via arquivo temporario.
- 2. Remocao fisica com reordenacao:** Dado um CPF, remova o registro **fisicamente** do arquivo (nao e exclusao logica) reconstruindo o arquivo via temporario — igual a insercao, mas pulando o registro a remover em vez de intercalar um novo.
- 3. Atualizacao (update) preservando ordem:** Dado um CPF, permita alterar nome/sobrenome/telefone/cidade. Como o CPF nao muda, a posicao relativa do registro no arquivo nao muda — mas como o novo conteudo pode ter tamanho diferente do antigo (nome maior, por exemplo), voce **nao pode sobrescrever in-place**. Precisa passar pelo arquivo temporario de novo.
- 4. Busca binaria adaptada:** Como os registros tem tamanho variavel, busca binaria "pura" por offset de byte nao funciona direto (nao da pra calcular $\text{meio} = \text{tamanho_arquivo}/2$ e cair exatamente no inicio de um registro). Implemente uma busca binaria **por indice de registro**: primeiro monte um indice em memoria (um array de long com os offsets de inicio de cada registro, percorrendo o arquivo uma vez sequencialmente), depois faca busca binaria nesse array de offsets, usando `fseek` para pular direto ao registro do meio a cada passo.
- 5. Merge de dois arquivos ordenados:** Dado `clientes.bin` e um segundo arquivo `importados.bin` (mesmo formato, tambem ordenado por CPF, sem duplicatas entre si nem internamente), implemente uma funcao que gere `clientes_merged.bin` com todos os registros dos dois arquivos, ordenados, **sem reler nenhum arquivo mais de uma vez inteiro** (faca leitura simultanea tipo merge de duas listas ordenadas — nao carregue tudo em memoria).

Pegadinha proposital

Na funcao de busca binaria por indice (item 4), o que acontece se, entre montar o indice e fazer a busca, alguem inserir ou remover um registro do arquivo (mudando os offsets)? Sua implementacao deve deixar claro (em comentario) que o indice e valido so para uma "foto" do arquivo naquele instante — e a funcao de busca deve recalculer o indice toda vez que for chamada, ou receber o indice ja pronto como parametro (pense no trade-off de cada abordagem).

Restricao de projeto: reaproveite as funcoes ja construidas (`pega_campo`, `pega_registro`, `grava_registro`) do exercicio anterior de insercao ordenada — nao reescreva a leitura de campos delimitados do zero.